

CAMPUS EVENT MANAGEMENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

Jammula Rithik Reddy (VTU25092)

10211CS224

Full Stack Application Development

Slot : E2



Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

MAY, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “Campus Event Management” by Jammula Rithik Reddy (vtu25092) has been carried out in the Winter Semester of Academic Year 2025–2026 (WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

MAY, 2026

Signature of Course Co-ordinator

Dr. Sumithra M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

MAY, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and, where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

(Jammula Rithik Reddy)

Date: 06/05/2026

ABSTRACT

The Campus Event Management System is a full-stack web application developed using Spring Boot, Spring MVC, Spring Security, Spring Data JPA, MySQL, and Thymeleaf. The system serves as a centralized digital platform for managing campus events, workshops, and seminars, enabling students and administrators to interact through a secure, role-based multi-user interface. Students can browse upcoming events, register for events with form validation, and provide feedback after attending. Administrators can create, update, and delete events, search events using filters such as date, department, and type, and view event registration statistics using aggregate functions. The platform integrates the **Email API** to send automated confirmation emails to students upon successful event registration. The project addresses United Nations Sustainable Development Goals (SDG 4, SDG 9) by promoting quality education, digital innovation, sustainable campus management, responsible consumption, and secure institutional processes. The system is hosted locally using a MySQL database and a Spring Boot embedded Tomcat server.

Keywords: Spring Boot, Spring MVC, Spring Security, JPA, MySQL, Thymeleaf, Email API, Event Management, Role-Based Access Control, REST API, Full Stack, Campus Portal

LIST OF FIGURES

1.1	Campus Event Management – Home Page	4
3.1	Spring Boot Project Structure	10
4.1	System Architecture Diagram – Spring Boot MVC with Email API .	15
4.2	Data Flow Diagram of the Campus Event Management System . . .	17
4.3	User Login and Registration Page	19
4.4	Event Listings Page with Search and Filter Options	21
4.5	Student Dashboard – Registered Events and Feedback	23
5.1	Admin Dashboard – Manage Events and View Statistics	26
5.2	Email Confirmation Sent via Email API on Event Registration . . .	27

LIST OF TABLES

5.1	Comparison of Campus Event Management System with Existing Platforms	28
6.1	Mapping of Project to Complex Engineering Problem (CEP) – Part 1	29
6.2	Mapping of Project to Complex Engineering Problem (CEP) – Part 2	30
6.3	Mapping of Project to Complex Engineering Activities (CEA) . . .	31
6.4	SDG Mapping for the Project	32

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
BCrypt	Blowfish Crypt (password hashing algorithm)
CEP	Complex Engineering Problem
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ES6	ECMAScript 2015
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
MVC	Model View Controller

MySQL	My Structured Query Language
ORM	Object Relational Mapping
PK	Primary Key
REST	Representational State Transfer
SDG	Sustainable Development Goal
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	7
3.1 Frontend Implementation	7
3.2 Backend Implementation	8
3.3 Database Implementation	11
4 METHODOLOGIES	14

4.1	Project Architecture	14
4.2	Data Flow Diagram	16
4.3	Module 1: User Authentication and Profile Management	18
4.4	Module 2: Event Management and Search	20
4.5	Module 3: Event Registration, Feedback, and Email Notification . .	22
5	RESULTS AND DISCUSSIONS	25
6	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	29
6.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	29
6.2	Mapping of the Project to Complex Engineering Activities (CEA) .	31
6.3	SDG Mapping (CEP Section)	32
7	CONCLUSION AND FUTURE ENHANCEMENTS	33
7.1	Conclusion	33
7.2	Future Enhancements	34
	References	34
	SOURCE CODE	37
1	Main Application Class	37
2	Event Controller	37
3	Event Service	38
4	Event Model	39

Chapter 1

INTRODUCTION

1.1 Introduction

The organisation of campus events has traditionally been a manual, time-consuming process for both students and administrators. With the rapid growth of digital technology, web-based management systems have become the primary means of streamlining institutional operations. The **Campus Event Management System** is a full-stack web application designed to automate and simplify the entire lifecycle of campus events, including workshops, seminars, cultural programmes, and technical competitions, on a single unified platform.

The system is built using the **Spring Boot** framework, which provides a production-ready backend infrastructure with embedded server support, auto-configuration, and seamless integration with Spring Security, Spring MVC, and Spring Data JPA. The frontend is rendered using **Thymeleaf** templates styled with HTML5 and CSS3. The application supports two distinct user roles, namely **Student** and **Admin**, each with a dedicated dashboard and role-specific functionalities. The **Email API** is integrated to dispatch automated confirmation and notification emails to students upon successful event registration.

1.2 Aim of the Project

The primary aim of this project is to develop a secure, scalable, and user-friendly campus event management web application that digitises the event registration and administration process. The system aims to:

- Provide students with a platform to browse upcoming events, register online, and submit feedback after attending.
- Enable administrators to create, edit, and delete events, and to search events using filters such as date, department, and event type.
- Integrate the Email API to automate student communication for event confirmations and cancellations.
- Implement role-based access control using Spring Security to ensure secure and segregated user experiences.
- Align the project with United Nations Sustainable Development Goals to address real-world educational and institutional challenges.

1.3 Scope of the Project

The scope of the Campus Event Management System encompasses the complete workflow for event organisation and participation within an academic context. The application covers:

- **User Management:** Registration, login, and role-based access control for Students and Administrators.
- **Event Management:** Creating, editing, deleting, and searching events with filters for category, department, date, and event type.

- **Registration Management:** Online event registration with form validation, duplicate prevention, and capacity tracking.
- **Feedback Management:** Submission of post-event feedback and ratings by registered students.
- **Email Integration:** Automated email notifications for registration confirmations and event cancellations.
- **Statistics:** Aggregate views of event registrations, attendance, and feedback ratings for admin reporting.
- **Security:** BCrypt password hashing, session management, and Spring Security role enforcement.
- **Database:** MySQL-backed persistence with JPA entities for Users, Events, Registrations, and Feedback.

The current system is designed for localhost deployment and does not require paid cloud infrastructure.

1.4 Framework

The project uses the following technical framework:

- **Backend:** Spring Boot 3.x, Spring MVC, Spring Security, Spring Data JPA, Hibernate ORM
- **Frontend:** Thymeleaf, HTML5, CSS3, JavaScript ES6
- **Database:** MySQL 8.0 (H2 for testing)
- **API:** Email API (JavaMailSender / SMTP)

- **Build Tool:** Maven (Spring Initializr)
- **IDE:** IntelliJ IDEA Community / VS Code
- **Server:** Embedded Apache Tomcat (Spring Boot default)

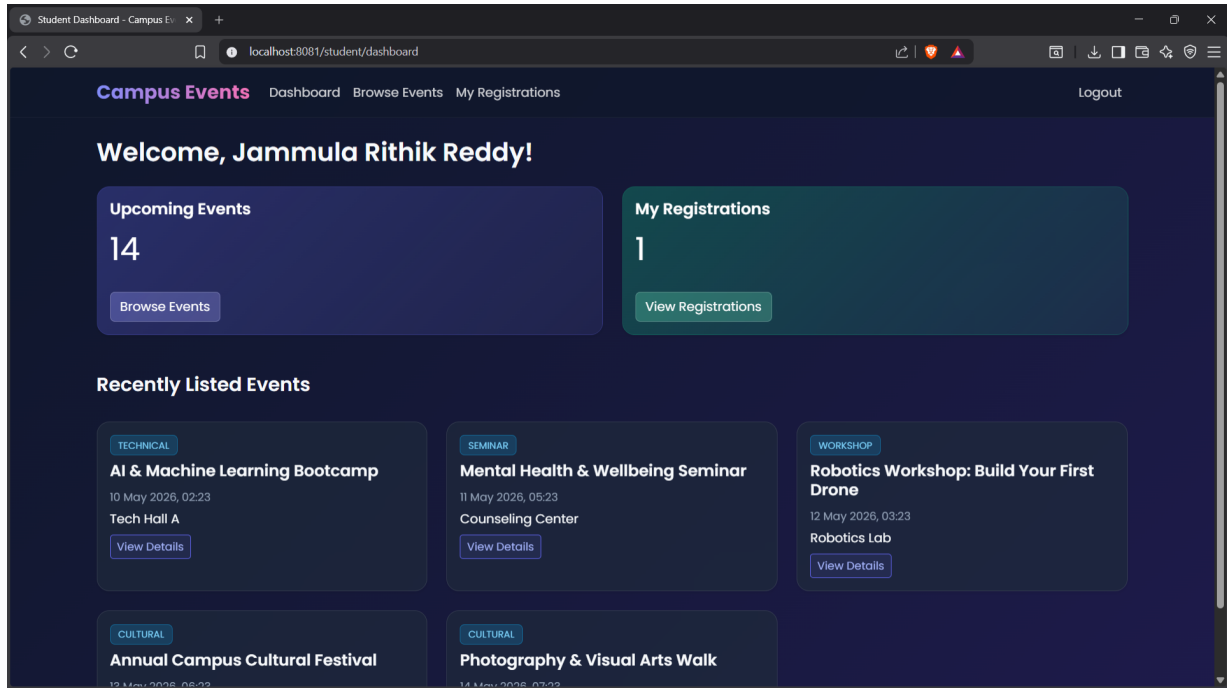


Figure 1.1: Campus Event Management – Home Page

Fig 1.1 illustrates the student dashboard (home page) of the Campus Event Management System. The interface provides a personalised welcome message along with a summary of key information such as the total number of upcoming events and the number of events the student has registered for. The dashboard includes quick navigation options like “Browse Events” and “View Registrations” to enhance user accessibility. Below the summary section, recently listed events are displayed in a card-based layout. Each event card presents essential details such as the event title, category (technical, seminar, workshop, or cultural), date, time, venue, and a “View Details” option for further interaction. This design ensures a user-friendly and visually organised experience, allowing students to efficiently explore and participate in campus events.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several event management platforms currently exist in the market, each addressing different aspects of digital event organisation. A thorough analysis of these platforms was conducted before designing the Campus Event Management System in order to identify gaps and opportunities for improvement.

Eventbrite (<https://www.eventbrite.com/>) is one of the most widely used event management platforms globally. It supports event creation, online ticketing, attendee management, and promotional tools. However, it is a commercial platform that charges fees for paid events, lacks role-based institutional control, and cannot be customised for an academic environment.

Google Forms with Google Sheets is a free and widely adopted combination for managing event registrations in academic institutions. It allows administrators to create registration forms and view responses in a spreadsheet. However, it does not support automated email notifications, role-based dashboards, or integrated feedback systems.

Microsoft Teams Events (<https://www.microsoft.com/en-us/microsoft-teams/>) provides virtual event scheduling, attendee registration, and meet-

ing integration. While suitable for online events, it does not support in-person campus event workflows, admin-level statistics, or departmental filtering.

Townscript (<https://www.townscript.com/>) is an Indian event management platform designed for academic and institutional events. It offers event listing, registration, and ticketing features. However, it is a third-party hosted platform and does not allow institutional branding or direct database access.

Unstop (formerly Dare2Compete) (<https://unstop.com/>) is a platform specifically designed for student competitions, hackathons, and academic events in India. It is highly relevant for student use but lacks the administrative control, custom filtering, and integrated feedback mechanisms required by institutions.

The Campus Event Management System differentiates itself from the platforms above by providing a fully customisable, open-source, role-based event management solution built with Spring Boot. It can be deployed locally without any cloud cost, making it an ideal solution for academic institutions and small organisations.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Campus Event Management System is built using **Thymeleaf** as the server-side templating engine, integrated directly with Spring Boot. No separate frontend server is required. The setup involves:

- Adding the `spring-boot-starter-thymeleaf` dependency in the Maven `pom.xml`.
- Placing all HTML templates in the `src/main/resources/templates/` directory.
- Placing static assets such as CSS, JavaScript, and images in the `src/main/resources/static/` directory.
- Using Thymeleaf's `th:` attributes to bind dynamic data from Spring MVC controllers to HTML views.

3.1.2 Structure of the Project

The frontend is organised as follows:

- `templates/` – Contains all Thymeleaf HTML files: `index.html`, `login.`, `register.`, `student/dashboard.html`, `student/events.html`, `student/event-detail.html`, `student/my-registrations.html`, `student/feedback.html`, `admin/dashboard.html`, `admin/`, `admin/event-form.html`, and `admin/statistics.html`.
- `static/css/` – Custom CSS stylesheets for responsive layout and dashboard styling.
- `static/js/` – JavaScript ES6 files for dynamic event filtering, form validation, and interactive UI behaviour.
- `static/images/` – Static image assets served by the embedded Tomcat server.

3.1.3 Execution Procedure

The frontend renders automatically when the Spring Boot application starts. Pages are served by the embedded Tomcat server at `http://localhost:8080`. Dynamic data such as event listings and registration statuses is injected into Thymeleaf templates by the corresponding Spring MVC controllers using `Model.addAttribute()`. JavaScript handles client-side interactions such as real-time event filtering and form validation before submission.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built with **Spring Boot 3.x** using Maven. The setup steps are:

- Generate the project at `https://start.spring.io` with dependencies: Spring Web, Spring Security, Spring Data JPA, Thymeleaf, MySQL Driver, Validation, Spring Boot Mail, and Spring Boot DevTools.

- Configure `application.properties` with the MySQL datasource URL, username, password, and JPA settings.
- Set `spring.jpa.hibernate.ddl-auto=update` for automatic schema creation on startup.
- Configure `spring.mail.host` and related SMTP properties for the Email API.

3.2.2 Structure of the Project

The backend follows a standard layered MVC architecture:

- `controller/` – Spring MVC `@Controller` classes handling HTTP requests for authentication, events, registrations, feedback, and admin operations.
- `service/` – Business logic layer: `UserService`, `EventService`, `RegistrationService`, `FeedbackService`, and `EmailService`.
- `repository/` – Spring Data JPA interfaces extending `JpaRepository` for all database operations.
- `model/` – JPA entity classes: `User`, `Event`, `Registration`, and `Feedback`.
- `security/` – Spring Security configuration for role-based access, BCrypt password encoding, and session management.
- `exception/` – Global exception handling using `@ControllerAdvice`.

3.2.3 Execution Procedure

To run the backend:

1. Start the MySQL server and create the database using: `CREATE DATABASE campus_events_db;`

2. Update `application.properties` with the correct MySQL credentials and SMTP email settings.
3. Run the Spring Boot application using `mvn spring-boot:run` or directly from IntelliJ IDEA or VS Code.
4. Access the application at `http://localhost:8080`.
5. Spring Boot will automatically create all required database tables on the first run via Hibernate DDL.

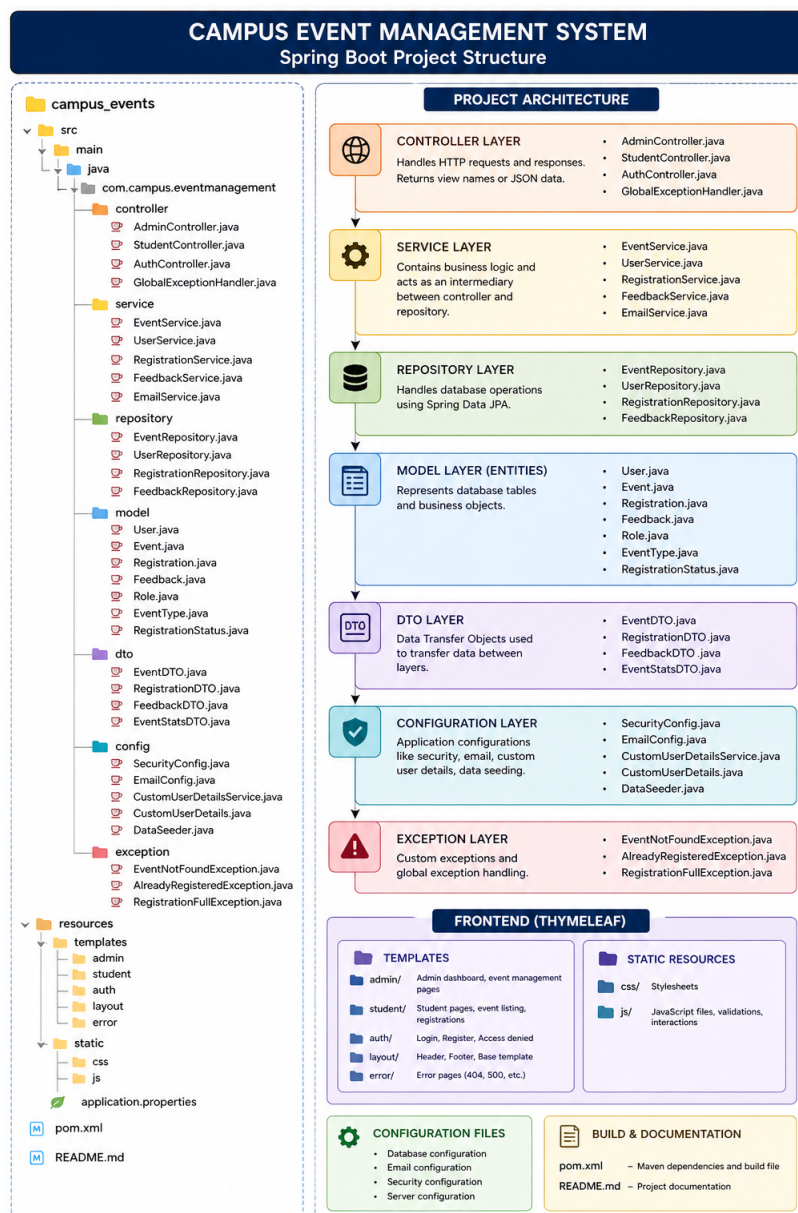


Figure 3.1: Spring Boot Project Structure

Figure 3.1 illustrates the overall project structure and layered architecture of the Campus Event Management System developed using Spring Boot. The project follows a modular and organised package structure under `src/main/java`, where the application is divided into multiple layers including Controller, Service, Repository, Model (Entities), DTO, Configuration, and Exception handling.

The **Controller layer** manages HTTP requests and routes them to appropriate services, while the **Service layer** contains the core business logic of the application. The **Repository layer** interacts with the database using Spring Data JPA for performing CRUD operations. The **Model layer** represents the database entities such as User, Event, Registration, and Feedback.

Additionally, the **DTO layer** is used for transferring data between layers efficiently, and the **Configuration layer** handles security, email setup, and custom user details. The **Exception layer** ensures proper handling of application-specific errors through custom exceptions and global handlers. The frontend implemented using Thymeleaf templates located in `resources/templates` directory, along with static assets such as CSS and JavaScript in the `resources/static` directory. Configuration files such as `application.properties` and build files like `pom.xml` are also included, ensuring a complete full-stack project structure.

3.3 Database Implementation

3.3.1 Database Configuration

The Campus Event Management System uses **MySQL 8.0** as the primary relational database, managed through **Spring Data JPA** and **Hibernate ORM**. The database connection configured in `src/main/resources/application.properties`:

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/campus_events_db
2 spring.datasource.username=root
3 spring.datasource.password=yourpassword
```

```
4 spring.jpa.hibernate.ddl-auto=update
5 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
6 spring.jpa.show-sql=true
```

The property `ddl-auto=update` ensures that Hibernate automatically creates or updates tables based on entity class definitions without dropping existing data. **H2 in-memory database** is used optionally during unit testing.

3.3.2 Schema Structure

The schema is organised around four primary JPA entities: **User**, **Event**, **Registration**, and **Feedback**. The `User` entity stores credentials and role information. The `Event` entity represents a campus event created by an administrator. The `Registration` entity links a student to a specific event and tracks status. The `Feedback` entity stores post-event ratings and comments submitted by students. All relationships are managed using JPA annotations: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@ManyToOne`, and `@OneToMany`.

3.3.3 Tables Created

The following tables are automatically created in the MySQL `campus_events_db` database:

- **users** – Stores: `id` (PK), `name`, `email`, `password` (BCrypt hashed), `role` (STUDENT/ADMIN), `depart`, `created_at`.
- **events** – Stores: `id` (PK), `title`, `description`, `event_type` (WORKSHOP/SEMINAR/CULTURAL/SPORTS/TECHNICAL), `department`, `event_date`, `venue`, `max_capacity`, `current_registrations`, `created_by` (FK → users), `created_at`.
- **registrations** – Stores: `id` (PK), `student_id` (FK → users), `event_id` (FK

→ events), registered_at, status (CONFIRMED / CANCELLED).

- **feedback** – Stores: id (PK), student_id (FK → users), event_id (FK → events), rating (1–5), comments, submitted_at.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Campus Event Management System follows a layered **MVC (Model-View-Controller)** architecture built on Spring Boot. The system consists of five layers communicating vertically, with one external API integration for email notifications.

Layer 1 – Client: The browser (Student or Admin) sends HTTP requests to the application server.

Layer 2 – Frontend: Thymeleaf templates render the user interface. JavaScript handles dynamic filtering and form validation on the client side. Spring Security intercepts all requests for authentication and role enforcement.

Layer 3 – Backend: Spring MVC `@Controller` classes, namely `AuthController`, `StudentController`, `AdminController`, and `GlobalExceptionHandler`, handle routing and request processing. Validation is enforced using `@NotNull`, `@Size`, and `@Valid` annotations. Global exceptions are handled by `@ControllerAdvice`.

Layer 4 – Data Access: Spring Data JPA repositories interact with the MySQL database. Custom JPQL queries handle event filtering by date, department, and type. Aggregate queries are used to generate registration statistics.

Layer 5 – External API: The Email API (JavaMailSender over SMTP) sends automated confirmation and cancellation emails to students from the backend EmailService.

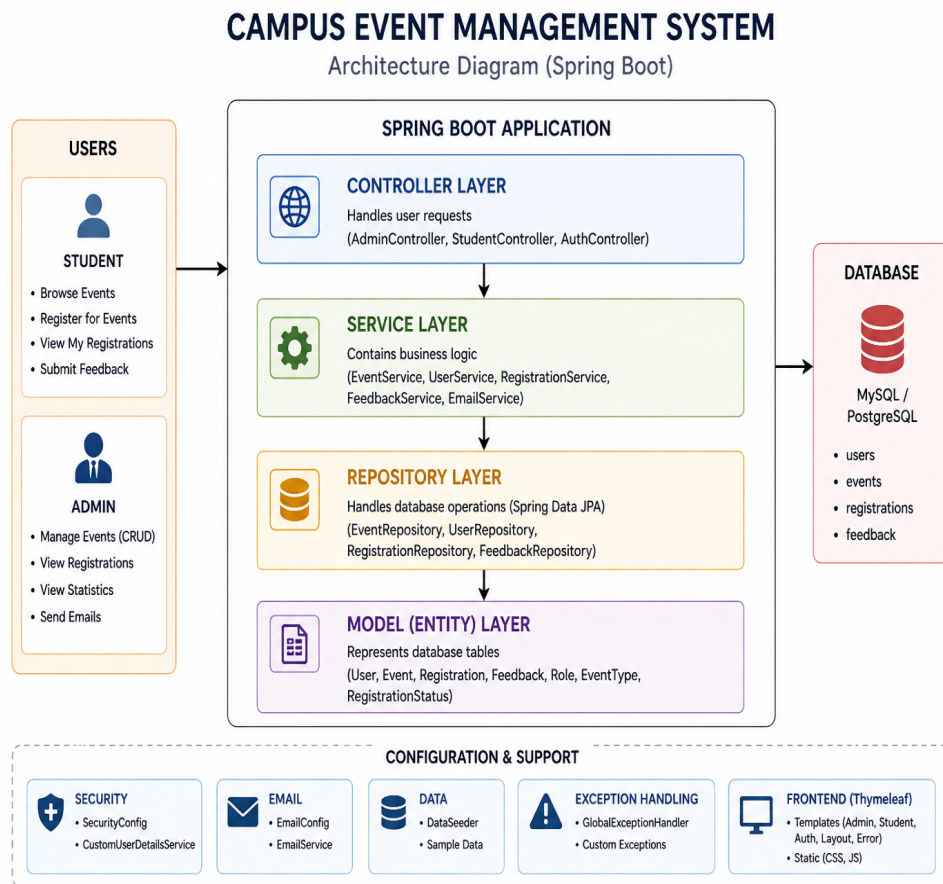


Figure 4.1: System Architecture Diagram – Spring Boot MVC with Email API

Figure 4.1 illustrates the system architecture of the Campus Event Management System based on the Spring Boot MVC framework. The architecture follows a layered design consisting of Controller, Service, Repository, and Model (Entity) layers, ensuring separation of concerns and maintainability.

The **Controller layer** handles incoming HTTP requests from users, including both students and administrators, and directs them to the appropriate services. The **Service layer** processes business logic such as event management, user handling, registration processing, feedback management, and email notifications. The **Repository layer** interacts with the database using Spring Data JPA to perform CRUD operations on entities such as users, events, registrations, and feedback. The **Model layer** represents

the database structure through entity classes.

The system supports two primary user roles: **Student** and **Admin**. Students can browse events, register, view their registrations, and submit feedback, while administrators can manage events, monitor registrations, view statistics, and send notifications.

The backend communicates with a relational database such as MySQL or PostgreSQL for persistent data storage. Additional supporting components include **Spring Security** for authentication and authorization, the **Email API** for sending automated notifications, data seeding for initial data setup, and global exception handling for managing application errors. The frontend is implemented using Thymeleaf templates along with static resources such as CSS and JavaScript, providing a complete full-stack architecture.

4.2 Data Flow Diagram

The data flow of the Campus Event Management System describes how information moves between users, the system, the database, and the Email API.

Student Flow: The student registers and logs in through Spring Security. After authentication, the student browses upcoming events fetched from the MySQL database. The student registers for an event by submitting a form that is validated on both the client and server side. Upon successful registration, the `EmailService` dispatches an automated confirmation email. The registration record is saved in the database with the status `CONFIRMED`. When an admin cancels an event, the student receives a cancellation notification email and the status is updated accordingly.

Admin Flow: The administrator logs in with admin credentials. The admin creates new events through a validated form, which saves the event to the `events` table. The admin can edit or delete events, apply search filters by date, department, or type, and

view registration statistics from aggregate database queries. On event cancellation, the system triggers email notifications to all registered students via the Email API.

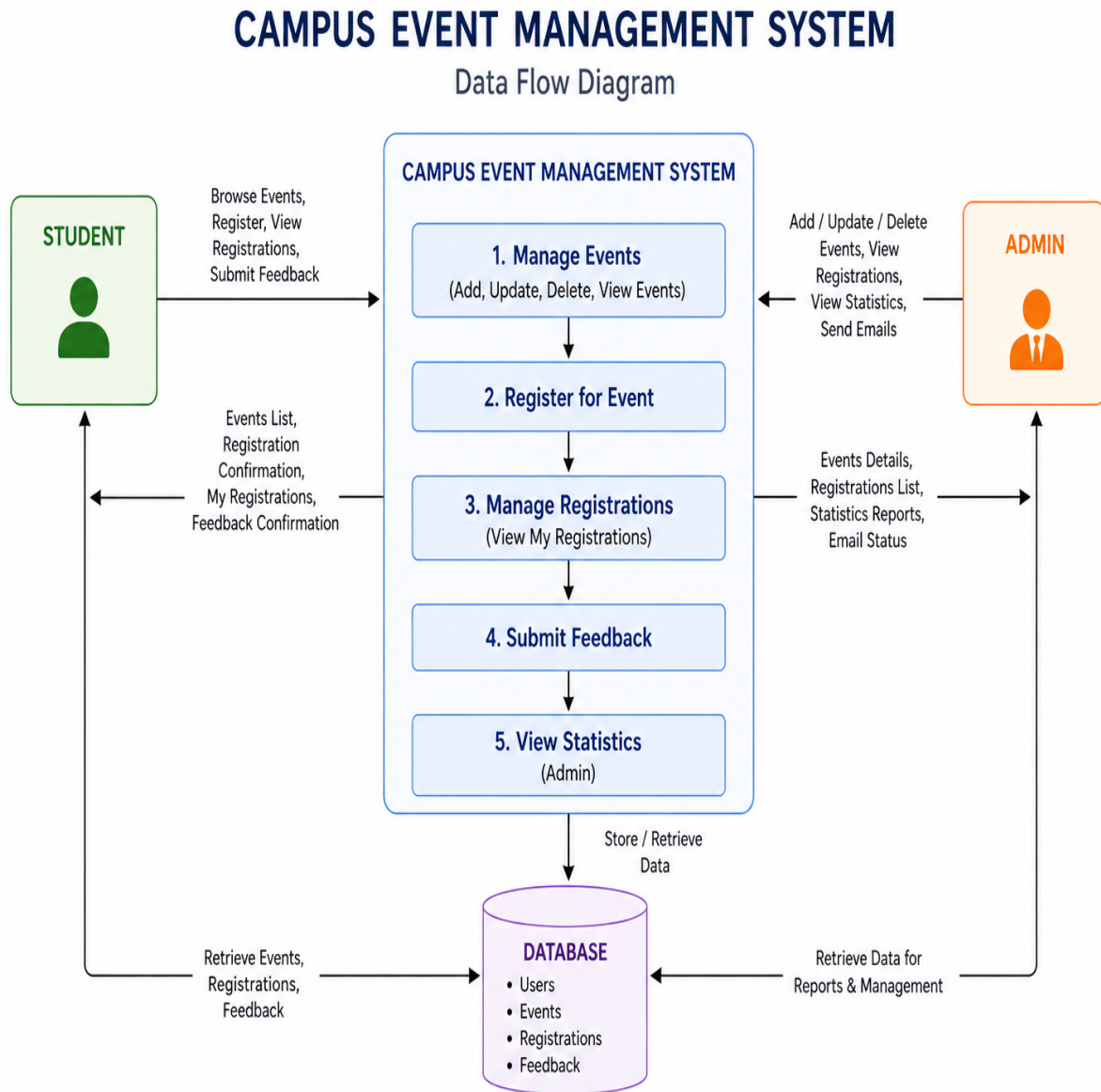


Figure 4.2: Data Flow Diagram of the Campus Event Management System

Figure 4.2 illustrates the data flow within the Campus Event Management System, highlighting the interaction between students, administrators, the application system, and the database. The system acts as a central processing unit where all operations are performed through a structured workflow.

Students interact with the system by browsing available events, registering for

events, viewing their registrations, and submitting feedback. These actions are processed by the system, which retrieves and stores relevant data in the database. Upon successful registration, confirmation details are generated and reflected in the student dashboard.

Administrators interact with the system to manage events by adding, updating, or deleting event details. They can also view student registrations, generate statistical reports, and send email notifications. The system processes these requests and communicates with the database to fetch or update the required information.

The database stores all essential data including users, events, registrations, and feedback. Data flows continuously between the system and the database for storing new records and retrieving existing information for display and reporting purposes. This structured flow ensures efficient data management and seamless interaction between all components of the system.

4.3 Module 1: User Authentication and Profile Management

This module handles all user-related operations using **Spring Security** and **BCrypt** password hashing.

Registration: New users fill in a registration form and select their role (Student or Admin). The `UserService` validates the input using `@Valid` and `@NotNull` annotations, hashes the password using `BCryptPasswordEncoder`, and saves the record to the `users` table via `UserRepository`.

Login: Spring Security's `AuthenticationManager` validates the submitted credentials against the hashed password stored in the database. On successful login, the user is redirected to their role-specific dashboard, either `/student/dashboard` or `/admin/dashboard`.

Session Management: Spring Security manages active user sessions. Logout

clears the session and redirects the user to the login page.

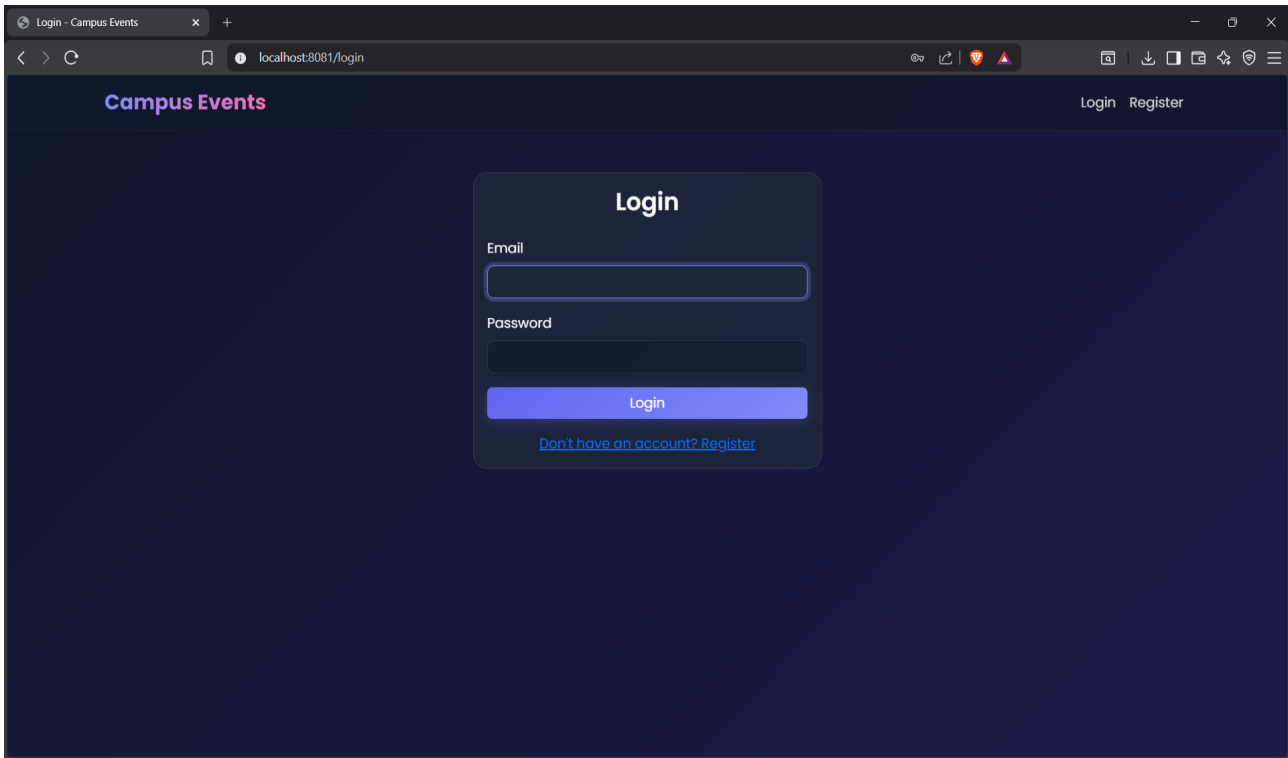


Figure 4.3: User Login and Registration Page

Figure 4.3 illustrates the user authentication interface of the Campus Event Management System, which includes both login and registration functionalities. The login page provides input fields for email and password, allowing registered users to securely access the system. The interface is designed with a clean and user-friendly layout to ensure ease of use.

The system uses Spring Security for authentication, where user credentials are validated against encrypted passwords stored in the database using BCrypt hashing. Upon successful login, users are redirected to their respective dashboards based on their roles, such as Student or Admin.

Additionally, new users can navigate to the registration page through the provided link, where they can create an account by submitting their details. This module ensures secure access control and acts as the entry point to the application.

4.4 Module 2: Event Management and Search

This module manages all event-related features for administrators and provides browsing capabilities for students.

Event CRUD Operations: Administrators create events by submitting a form that captures the event title, description, type, department, date, venue, and maximum capacity. The `AdminController` saves the event to the `events` table via `EventRepository`. Admins can edit or delete any event. Deletion triggers email notifications to all registered students.

Event Search and Filtering: The admin panel provides filters for event type, department, and date range. These filters are implemented using custom JPQL finder methods in `EventRepository`, such as `findByEventType()`, `findByDepartment()`, and `findByEventDateBetween()`.

Student Event Browsing: Students view a list of upcoming events sorted by date. Event cards display the title, date, venue, type badge, and a capacity progress bar. The Register button is disabled if the event is at full capacity or if the student has already registered.

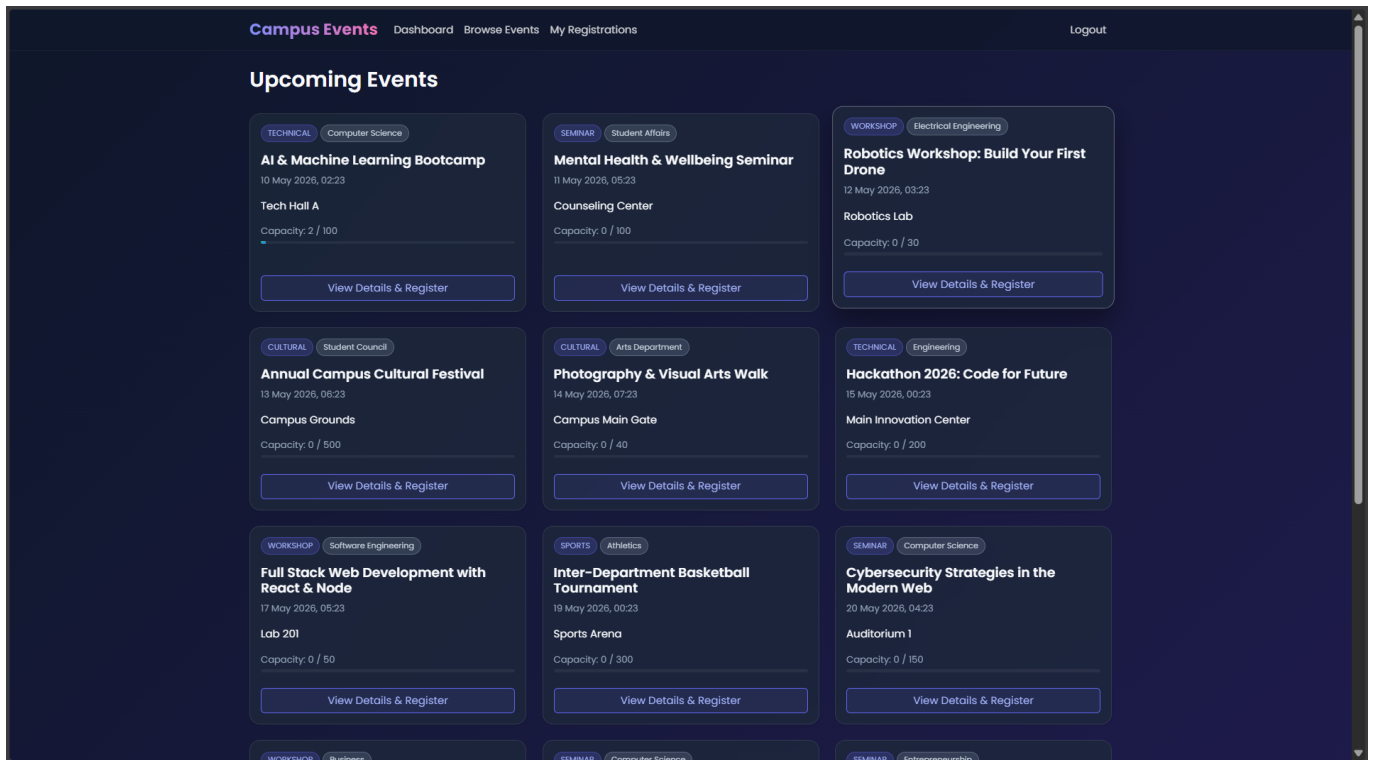


Figure 4.4: Event Listings Page with Search and Filter Options

Figure 4.4 illustrates the event listings page of the Campus Event Management System, where students can view all upcoming events in a structured and visually appealing format. Events are displayed as individual cards containing essential details such as event title, category, department, date, time, venue, and current registration capacity.

Each event card includes a progress indicator showing the number of registered participants relative to the maximum capacity, helping users quickly identify availability. The “View Details & Register” button allows students to access more information about the event and proceed with registration.

The page supports search and filtering functionality, enabling users to sort events based on criteria such as department, event type, and date. This feature enhances usability by allowing students to easily find relevant events according to their interests. The responsive card-based layout ensures an organised user interface and improves overall user experience.

4.5 Module 3: Event Registration, Feedback, and Email Notification

This module handles the student registration workflow, feedback collection, and automated email communication.

Event Registration: When a student clicks “Register” and submits the form, the `RegistrationService` checks for duplicate registrations and verifies that the event has not reached its maximum capacity. If all conditions are satisfied, the registration is saved to the `registrations` table with status `CONFIRMED` and the `current_registrations` count is incremented.

Email API Integration: Immediately after a successful registration, the `EmailService` uses `JavaMailSender` to send a confirmation email to the student:

```
1 Subject: Event Registration Confirmed – {eventTitle}
2 Body: Dear {studentName},
3 You have successfully registered for {eventTitle}
4 on {eventDate} at {venue}.
5 We look forward to seeing you!
```

On event cancellation by the admin, a cancellation email is dispatched to all registered students.

Feedback Submission: After an event has taken place, registered students can submit a feedback form with a rating from 1 to 5 and optional comments. The `FeedbackService` saves the record to the `feedback` table. Admins can view aggregate ratings per event from the statistics page.

Status	Event Name	Date	Venue	Action
CONFIRMED	AI & Machine Learning Bootcamp	10 May 2026, 02:23 AM	Tech Hall A	View
CONFIRMED	Mental Health & Wellbeing Seminar	11 May 2026, 05:23 AM	Counseling Center	View
CONFIRMED	Robotics Workshop: Build Your First Drone	12 May 2026, 03:23 AM	Robotics Lab	View
CONFIRMED	Photography & Visual Arts Walk	14 May 2026, 07:23 AM	Campus Main Gate	View
CONFIRMED	Photography & Visual Arts Walk	14 May 2026, 07:23 AM	Campus Main Gate	View

Figure 4.5: Student Dashboard – Registered Events and Feedback

Figure 4.5 illustrates the “My Registrations” section of the student dashboard in the Campus Event Management System. This page provides a structured view of all events for which the student has successfully registered. The information is presented in a tabular format, including details such as registration status, event name, date, venue, and available actions.

Each record displays the current status of the registration, typically marked as CONFIRMED, ensuring clarity for the user. The “View” action allows students to access detailed information about the selected event. This module enables students to efficiently track their participation in various campus activities.

The design ensures a clean and organised layout, improving readability and usability. It also supports easy navigation and helps students manage their event engagements effectively within the system. **Advantages of this Project**

The Campus Event Management System provides significant advantages for all

stakeholders involved in campus event organisation.

a) Automated and Efficient Event Management: The integration of Spring Boot REST APIs and the Email API automates event registration and communication, eliminating manual processes for both students and administrators. Students receive instant email confirmations, significantly reducing communication delays compared to traditional paper-based workflows.

b) Secure and Role-Based Access Control: Spring Security ensures that each user role, namely Student and Admin, can only access their designated features and pages. BCrypt password hashing protects all user credentials, and session management prevents unauthorised access across all user actions.

c) Scalable Full-Stack Architecture: The separation of concerns through MVC layered architecture, combined with Spring Data JPA and MySQL, ensures that the system can scale to accommodate a large number of concurrent users, events, and registrations without degradation in performance or security.

Disadvantages

Despite its strengths, the system has certain limitations. The Email API depends on an SMTP server, which requires valid credentials and may be subject to sending limits on free SMTP providers such as Gmail. The current system is designed for localhost deployment and does not support cloud hosting without additional configuration. The system does not include a real-time notification mechanism; instead, it relies on email-based communication. Additionally, the system does not currently incorporate an AI-powered event recommendation engine, which would improve student engagement and participation.

Chapter 5

RESULTS AND DISCUSSIONS

The Campus Event Management System was successfully developed, tested, and verified in a localhost environment using Spring Boot, MySQL, Thymeleaf, and the Email API. Both user roles, namely Student and Admin, were tested independently to validate role-based access, CRUD operations, search and filtering, and email integration.

Authentication Testing: User registration and login were tested for both Student and Admin roles. Spring Security correctly redirected authenticated users to their role-specific dashboards and denied access to unauthorised routes.

Event Module Testing: Administrators successfully created, edited, and deleted events. Students browsed and filtered events using type, department, and date filters. Event capacity tracking was validated by disabling the register button when an event reached its maximum capacity.

Registration and Email Testing: Students registered for events and received automated confirmation emails within 30 seconds of submission. The student dashboard correctly reflected all registered events with their current status. Cancellation emails were verified after an admin deleted an event with registered students.

Feedback and Statistics Testing: Students submitted post-event feedback with ratings. The admin statistics page correctly displayed registration counts, fill percentages,

and average ratings per event using aggregate JPQL queries.

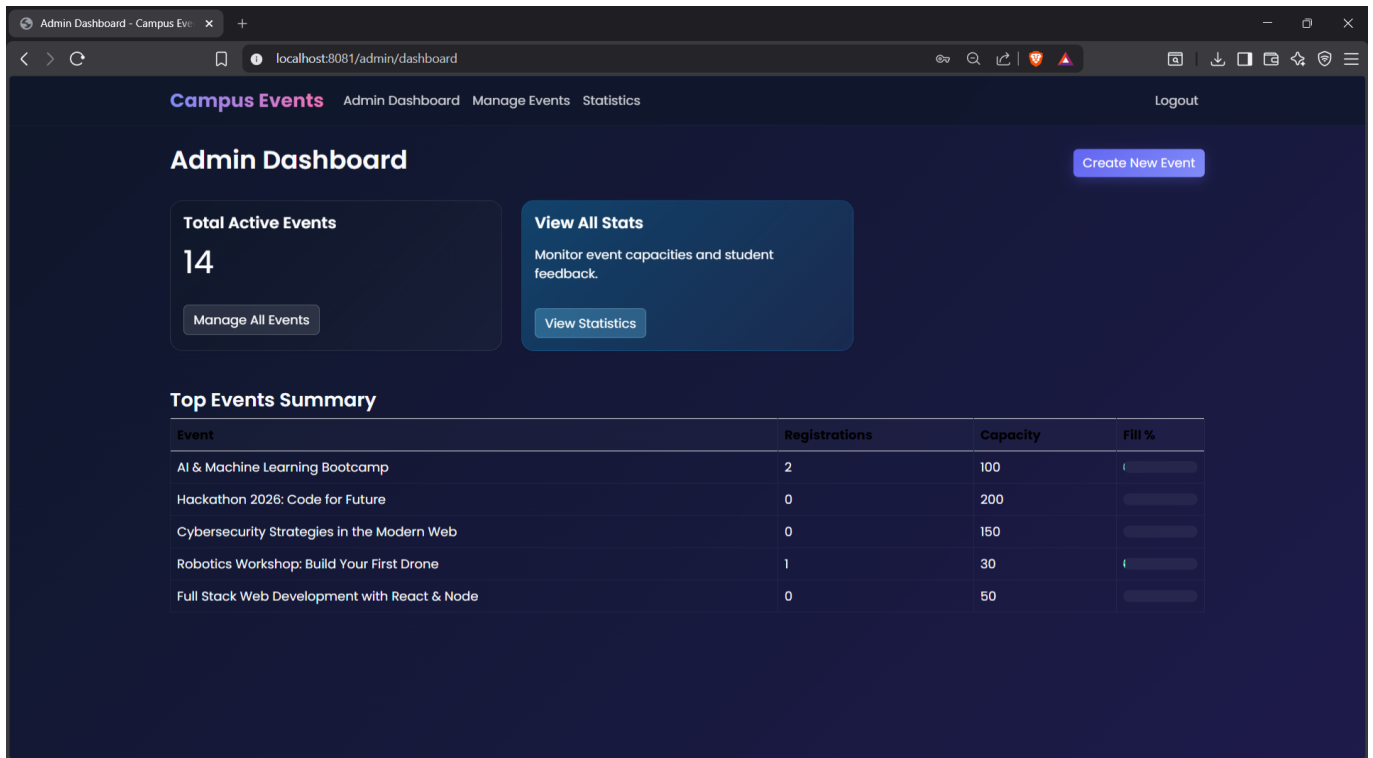


Figure 5.1: Admin Dashboard – Manage Events and View Statistics

Figure 5.1 illustrates the admin dashboard of the Campus Event Management System, which provides a comprehensive overview of event management and system statistics. The dashboard displays key information such as the total number of active events and quick navigation options for managing events and viewing statistical data.

Administrators can create new events using the “Create New Event” option and manage existing events through the “Manage All Events” functionality. The “View Statistics” section enables administrators to monitor event performance, including participant registrations and feedback.

The dashboard also includes a summary table of top events, presenting details such as event name, number of registrations, maximum capacity, and fill percentage. This allows administrators to easily analyse event participation and make informed decisions for future planning.

Overall, the admin dashboard provides a centralised interface for efficient event

management, monitoring, and decision-making within the system.

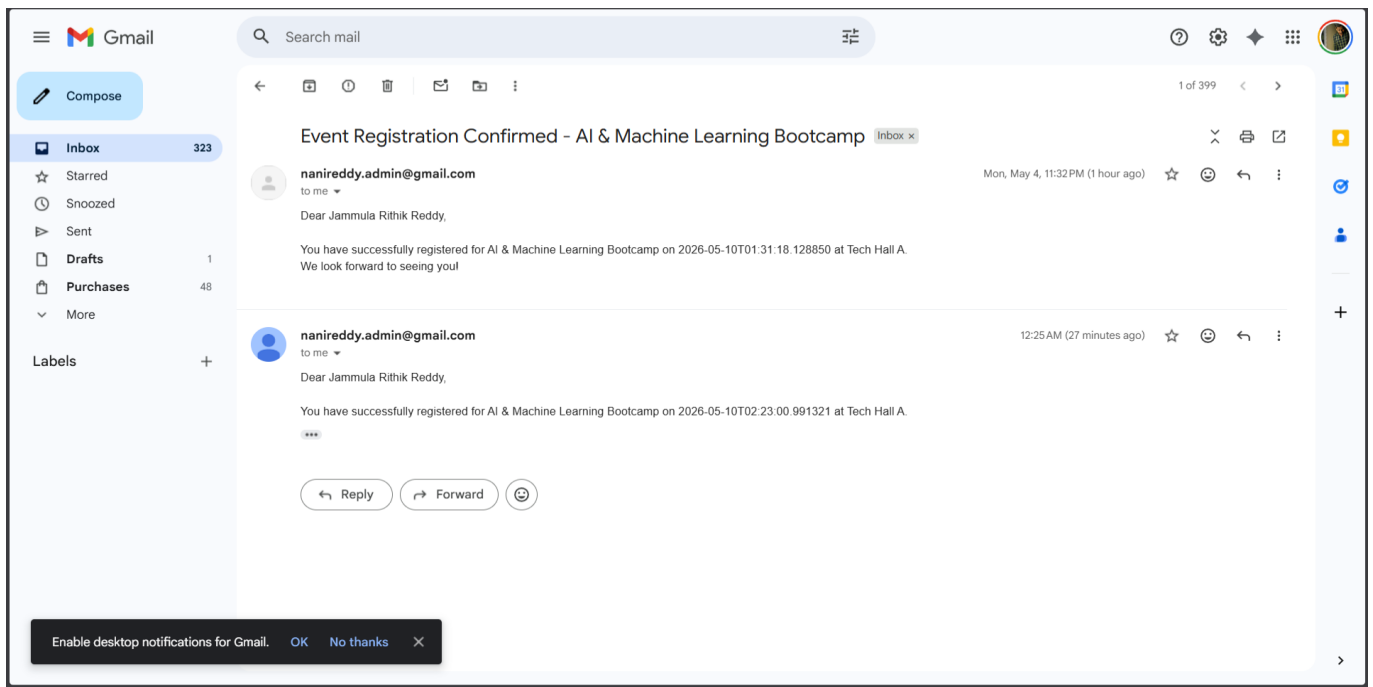


Figure 5.2: Email Confirmation Sent via Email API on Event Registration

Figure 5.2 illustrates the automated email confirmation sent to a student upon successful event registration in the Campus Event Management System. The email includes essential details such as the event title, date, time, and venue, ensuring that the user receives immediate confirmation of their registration.

This functionality is implemented using the Email API integrated with the backend through JavaMailSender and SMTP configuration. Once a student registers for an event, the system triggers the email service, which generates and sends a personalised confirmation message to the registered email address.

The automated notification system enhances user experience by providing timely communication and reduces the need for manual follow-ups. It also ensures reliability and transparency in the event registration process by keeping users informed about their participation.

Table 5.1: Comparison of Campus Event Management System with Existing Platforms

Feature	This System	Eventbrite	Google Forms	Townscript
Role-Based Access	Yes	Partial	No	Partial
Event Feedback System	Yes	No	No	No
Email Notifications	Yes (SMTP)	Yes	No	Yes
Admin Statistics Panel	Yes	Partial	No	Partial
Custom Event Filters	Yes	Partial	No	Partial
Free to Deploy	Yes	No	Yes	No
Custom Admin Panel	Yes	No	No	No
Spring Boot Backend	Yes	No	No	No
Open Source	Yes	No	No	No

Chapter 6

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

6.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 6.1: Mapping of Project to Complex Engineering Problem (CEP) – Part 1

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge of Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies such as Thymeleaf and HTML/CSS for developing a scalable event management system.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and robust data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.

The above table demonstrates how the project addresses key attributes of Complex Engineering Problems (CEP), including depth of knowledge, conflicting requirements, and analytical design. It highlights the integration of modern technologies such as

Spring Boot, database systems, and secure architectures. The system effectively balances usability, security, and scalability while solving real-world institutional challenges.

Table 6.2: Mapping of Project to Complex Engineering Problem (CEP) – Part 2

Level	Attribute	Characteristics	WKS	Justification
WP4	Familiarity	Partially new problems	WK4, WK8	Integrating real-time event updates, user interaction tracking, and admin analytics introduces challenges that go beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation that is not fully covered by standard Spring Boot templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Addresses the distinct requirements of students, faculty coordinators, and administrators, ensuring smooth event organisation and participation.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates the frontend UI, backend services, database, validation, email notification, and security modules into one unified system.

This table further explains additional CEP attributes such as familiarity, stakeholder needs, and system interdependence. The project involves multiple stakeholders and integrates various modules including frontend, backend, database, and email services. It showcases the ability to design a comprehensive system that meets diverse requirements and operates as a unified solution.

6.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 6.3: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	The project uses Spring Boot, Spring MVC, Spring Data JPA, MySQL, Thymeleaf, Spring Security, and Java-MailSender for full-stack event management development.
EA2	Level of Interactions	Complex interactions between modules	The system manages layered interactions between event management, user registration, form validation, authentication, email notification, and admin reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralised platform with dynamic event filtering, real-time capacity tracking, automated email notifications, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency by reducing manual event management errors and communication delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, data validation, email integration, and data analytics into a unified system that extends beyond traditional event handling methods.

This table further explains additional CEP attributes such as familiarity, stakeholder needs, and system interdependence. The project involves multiple stakeholders and integrates various modules including frontend, backend, database, and email services. It showcases the ability to design a comprehensive system that meets diverse requirements and operates as a unified solution.

6.3 SDG Mapping (CEP Section)

Table 6.4: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning and skill development beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralised, automated event management system built on modern web technologies.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitising event processes, improving resource management, and reducing administrative overhead.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling fully online event registration, management, and communication, thereby supporting environmentally responsible institutional practices.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through role-based authentication, Spring Security access control, and BCrypt password hashing.

This table presents the alignment of the project with various United Nations Sustainable Development Goals (SDGs). The system contributes to quality education, digital innovation, and sustainable campus management. It also promotes responsible resource usage and secure institutional processes through its fully digital and role-based architecture.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The Campus Event Management System successfully addresses a real-world complex engineering problem by developing a secure, multi-role, full-stack web application that streamlines the organisation and participation of campus events on a unified digital platform. The project demonstrates the practical application of Spring Boot, Spring MVC, Spring Security, Spring Data JPA, MySQL, and Thymeleaf in building an enterprise-grade academic management system. The integration of the **Email API** automates student communication for registration confirmations and event cancellations without requiring a separate mail server setup. All core features, including user authentication, event creation and management, online registration with capacity tracking, post-event feedback, and admin statistics, have been implemented and tested successfully on a localhost environment. The system aligns with United Nations Sustainable Development Goals SDG 4, SDG 9, SDG 11, SDG 12, and SDG 16 by promoting quality education, digital innovation, sustainable campus management, responsible consumption, and secure institutional processes. All CEP attributes, including depth of knowledge, analytical thinking, multi-stakeholder handling, and system-level integration, have been thoroughly addressed throughout the

project lifecycle.

7.2 Future Enhancements

Several enhancements are planned to further improve the system in future iterations:

- **AI-Powered Event Recommendations:** Integrating a machine learning recommendation engine to suggest relevant events to students based on their department, interests, and registration history.
- **Cloud Deployment:** Deploying the application on AWS, Heroku, or Azure to make it publicly accessible beyond a localhost environment, with auto-scaling support for high-traffic periods.
- **Real-Time Notifications:** Replacing the current email-based notification system with WebSocket or Firebase push notifications for instant event updates.
- **QR Code-Based Attendance:** Generating and scanning QR codes for physical event check-in, enabling automated attendance tracking and reporting.
- **Mobile Application:** Building a companion mobile application using React Native or Flutter to extend the platform's reach to smartphone users.
- **Calendar Integration:** Integrating with Google Calendar or Outlook to allow students to add registered events directly to their personal calendars with one click.
- **Advanced Analytics Dashboard:** Providing graphical charts and visual reports for event trends, department-wise participation, and semester-wise event statistics.

References

- [1] Eventbrite Event Management Platform. Available at: <https://www.eventbrite.com/>. Eventbrite provides tools for event creation, ticketing, attendee management, and promotions. Accessed April 2026.
- [2] Townscript – Online Event Management. Available at: <https://www.townscript.com/>. Townscript is an Indian event management platform supporting academic and institutional event registration. Accessed April 2026.
- [3] Unstop (formerly Dare2Compete). Available at: <https://unstop.com/>. Unstop is a student-focused platform for competitions, hackathons, and academic events in India. Accessed April 2026.
- [4] Spring Boot Documentation, VMware Tanzu. Available at: <https://spring.io/projects/spring-boot>. Accessed April 2026.
- [5] Spring Security Reference Documentation. Available at: <https://docs.spring.io/spring-security/reference/>. Accessed April 2026.
- [6] Spring Data JPA Reference Documentation. Available at: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>. Accessed April 2026.
- [7] Spring Boot Mail Starter Documentation. Available at: <https://docs.spring.io/spring-boot/docs/current/reference/html/io.html#io.email>. Accessed April 2026.

- [8] MySQL 8.0 Reference Manual, Oracle Corporation. Available at: <https://dev.mysql.com/doc/refman/8.0/en/>. Accessed April 2026.
- [9] Thymeleaf Template Engine Documentation. Available at: <https://www.thymeleaf.org/documentation.html>. Accessed April 2026.
- [10] Apache Maven Documentation. Available at: <https://maven.apache.org/guides/>. Accessed April 2026.
- [11] United Nations Sustainable Development Goals. Available at: <https://sdgs.un.org/goals>. Accessed April 2026.
- [12] Hibernate ORM Documentation. Available at: <https://hibernate.org/orm/documentation/>. Accessed April 2026.

SOURCE CODE

1 Main Application Class

```
1 package com.campus.eventmanagement;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class EventManagementApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(EventManagementApplication.class, args);
11     }
12 }
```

Listing 7.1: Spring Boot Main Class

2 Event Controller

```
1 package com.campus.eventmanagement.controller;
2
3 import com.campus.eventmanagement.model.Event;
4 import com.campus.eventmanagement.service.EventService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @Controller
13 @RequestMapping("/events")
14 public class EventController {
15
16     @Autowired
17     private EventService eventService;
```

```

18
19 @GetMapping
20 public String getAllEvents(Model model) {
21     List<Event> events = eventService.getAllEvents();
22     model.addAttribute("events", events);
23     return "events";
24 }
25
26 @GetMapping("/create")
27 public String showCreateForm(Model model) {
28     model.addAttribute("event", new Event());
29     return "create-event";
30 }
31
32 @PostMapping("/save")
33 public String saveEvent(@ModelAttribute Event event) {
34     eventService.saveEvent(event);
35     return "redirect:/events";
36 }
37
38 @GetMapping("/delete/{id}")
39 public String deleteEvent(@PathVariable Long id) {
40     eventService.deleteEvent(id);
41     return "redirect:/events";
42 }
43 }

```

Listing 7.2: Event Controller Class

3 Event Service

```

1 package com.campus.eventmanagement.service;
2
3 import com.campus.eventmanagement.model.Event;
4 import com.campus.eventmanagement.repository.EventRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service
11 public class EventService {
12
13     @Autowired
14     private EventRepository eventRepository;
15
16     public List<Event> getAllEvents() {

```

```

17         return eventRepository.findAll();
18     }
19
20     public void saveEvent(Event event) {
21         eventRepository.save(event);
22     }
23
24     public void deleteEvent(Long id) {
25         eventRepository.deleteById(id);
26     }
27 }

```

Listing 7.3: Event Service Class

4 Event Model

```

1 package com.campus.eventmanagement.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDate;
5
6 @Entity
7 @Table(name = "events")
8 public class Event {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String title;
15     private String description;
16     private String department;
17     private String eventType;
18     private LocalDate eventDate;
19     private String venue;
20     private int maxCapacity;
21
22     public Event() {}
23
24     public Event(String title, String description, String department,
25                 String eventType, LocalDate eventDate,
26                 String venue, int maxCapacity) {
27         this.title = title;
28         this.description = description;
29         this.department = department;
30         this.eventType = eventType;
31         this.eventDate = eventDate;

```

```

32         this.venue = venue;
33         this.maxCapacity = maxCapacity;
34     }
35
36     public Long getId() { return id; }
37
38     public String getTitle() { return title; }
39
40     public void setTitle(String title) { this.title = title; }
41
42     public String getDescription() { return description; }
43
44     public void setDescription(String description) { this.description =
45         description; }
46
47     public String getDepartment() { return department; }
48
49     public void setDepartment(String department) { this.department = department;
50     }
51
52     public String getEventType() { return eventType; }
53
54     public void setEventType(String eventType) { this.eventType = eventType; }
55
56     public LocalDate getEventDate() { return eventDate; }
57
58     public void setEventDate(LocalDate eventDate) { this.eventDate = eventDate;
59     }
60
61     public String getVenue() { return venue; }
62
63     public void setVenue(String venue) { this.venue = venue; }
64
65     public int getMaxCapacity() { return maxCapacity; }
66
67     public void setMaxCapacity(int maxCapacity) { this.maxCapacity = maxCapacity
68     ; }
69 }

```

Listing 7.4: Event Entity Class